

Práctica 5: Kubernetes y Raft



**Escuela de
Ingeniería y Arquitectura
Universidad Zaragoza**

**Rubén Agustín Galdeano (820829)
Antonio González Almela (143045)**

Contenido

1.- Introducción	3
2.- Decisiones de implantación	3
2.1 Usamos: StatefulSet (con Headless Service).	3
2.2.- Service / descubrimiento	3
2.3.- Cómo pasar la configuración (lista de peers y puertos)	3
3.- Arquitectura de Kubernetes.....	5
4.- Anexos.....	5
4.1.- raft-statefulset.yaml	5
4.2 cliente-pod.yaml	7

1.- Introducción

En esta práctica configuramos un sistema distribuido que ejecuta tres nodos Raft en un entorno Kubernetes. Para ello hemos encapsulado en contenedores Docker el código ejecutable que despliega un nodo Raft. Una vez creada la estructura Kubernetes con un 'Control Plane', tres workers y un cliente, hemos definido los manifiestos (.yaml) que permiten poner en ejecución en cada worker un nodo raft y en el cliente un código que sincroniza el inicio de los nodos Raft.

2.- Decisiones de implantación

2.1 Usamos: StatefulSet (con Headless Service).

Por qué: Raft necesita identidad estable (nombre/orden) y posibilidades de almacenamiento persistente por réplica (un PV/PVC por réplica). StatefulSet da nombres DNS estables name-0, name-1, ... y facilita el uso de PVCs únicos por réplica.

Si optásemos por la alternativa Deployment/ReplicaSet — no podríamos garantizar la identidad estable por réplica.

2.2.- Service / descubrimiento

Crear un Headless Service (clusterIP: None) para que cada Pod del StatefulSet tenga DNS propio svcname-N.namespace.svc.cluster.local. Los nodos Raft podrán conocerse por esa DNS.

2.3.- Cómo pasar la configuración (lista de peers y puertos)

En el StatefulSet del despliegue Raft, cada pod ejecuta el contenedor nodoraft. Este contenedor recibe los argumentos de inicio directamente a través del campo args: del manifiesto, que Kubernetes entrega al proceso como parámetros de la línea de comandos (os.Args en Go).

```
args:
  - "$(MY_ID)"
  - "raft-0.raft-svc:29280"
  - "raft-1.raft-svc:29280"
  - "raft-2.raft-svc:29280"
env:
  - name: MY_ID
```

```
valueFrom:  
  fieldRef:  
    fieldPath: metadata.name
```

Kubernetes define la variable de entorno MY_ID:

```
env:  
- name: MY_ID  
  valueFrom:  
    fieldRef:  
      fieldPath: metadata.name
```

2.4.- Imágenes y registry

Construid imágenes Docker para servidor y clientes. Push a registry.

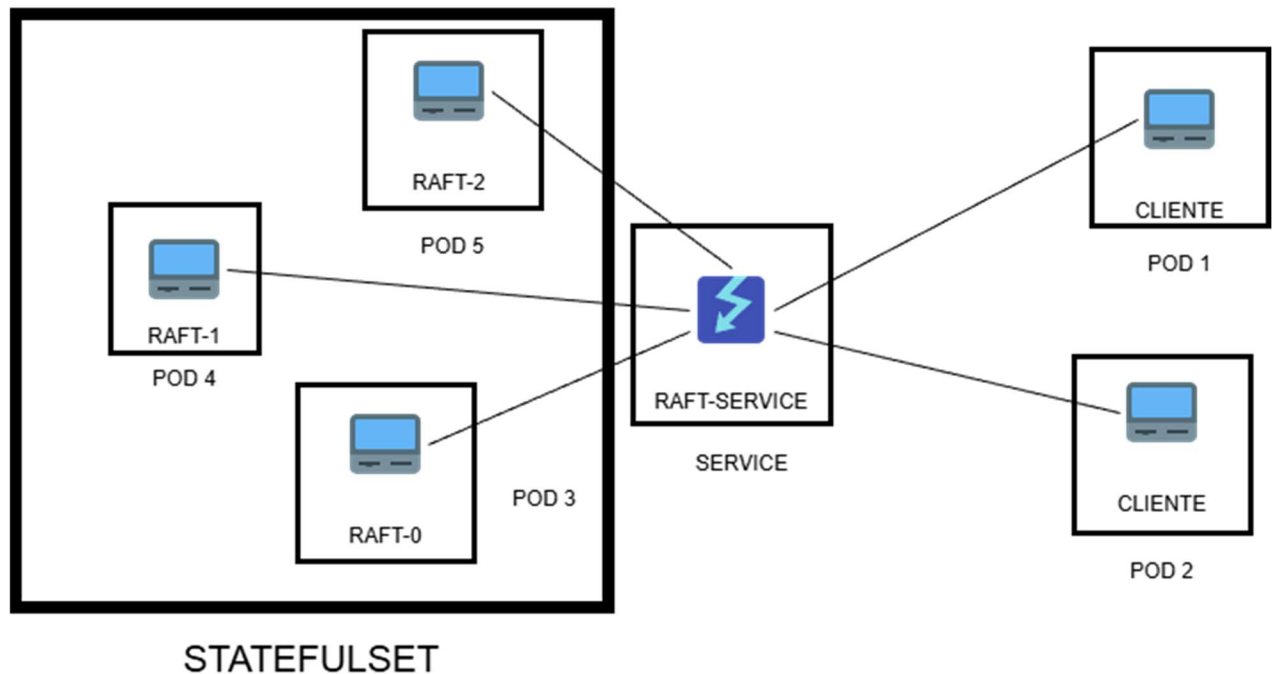
2.5 Verificación de nodos listos (readiness/liveness)

Para comprobar si el RPC server está escuchando hemos implementado un método RPC de “Ping”, de forma que es muy ligero y nos permite sincronizar cuando los nodos Raft ya han creado su log y están listos para responder a peticiones RPC. De esta forma Kubernetes no enviará tráfico a Pods que aún no están listos.

2.6 Servicios de cliente

Usamos Job para ejecutar el cliente que someterá operaciones al nodo líder.

3.- Arquitectura de Kubernetes



4.- Anexos

4.1.- raft-statefulset.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: raft-svc
spec:
  clusterIP: None
  selector:
    app: raft
  ports:
    - name: rpc
      port: 29280
      protocol: TCP
```

```
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: raft
spec:
  serviceName: "raft-svc"
  replicas: 3
  selector:
    matchLabels:
      app: raft
  template:
    metadata:
      labels:
        app: raft
    spec:
      containers:
        - name: raft-node
          image: localhost:5001/nodoraft:latest
          imagePullPolicy: IfNotPresent
          # command: ["/bin/sh", "-c"]
          # args:
            # - - "/.nodoRaft \"$MY_ID\" raft-0.raft-svc:29280 raft-1.raft-svc:29280 raft-2.raft-
svc:29280"
          args:
            - "$(MY_ID)"
            - "raft-0.raft-svc:29280"
            - "raft-1.raft-svc:29280"
            - "raft-2.raft-svc:29280"
          env:
            - name: MY_ID
              valueFrom:
                fieldRef:
                  fieldPath: metadata.name
```

```
ports:
- containerPort: 29280
  name: rpc
readinessProbe:
  tcpSocket:
    port: rpc
  initialDelaySeconds: 3
  periodSeconds: 3
  failureThreshold: 10
livenessProbe:
  tcpSocket:
    port: rpc
  initialDelaySeconds: 30
  periodSeconds: 10
# opcional: intentar distribuir pods en distintos nodos (si el cluster tiene varios workers)
affinity:
  podAntiAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchLabels:
            app: raft
        topologyKey: "kubernetes.io/hostname"
```

4.2 cliente-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: cliente
spec:
  containers:
  - name: cliente
    image: localhost:5001/cliente:latest
    args:
    - "0"
```

- "raft-0.raft-svc:29280"
- "raft-1.raft-svc:29280"
- "raft-2.raft-svc:29280"